



Tunis Business School
Business Analytics – Data Mining Project

IMDB Movie Reviews Sentiment Analysis

Authors:

Yassine Saadi
Sarah Mneja
Bahe Amri
Sirine Zaltani

Supervised by:

Professor: Lassaad El Moubarki

2025–2026

Contents

1	Introduction	3
1.1	Data Source and Collection	3
1.2	Data Description	3
1.3	Data Size	3
1.4	Exploratory Data Analysis (EDA)	3
1.4.1	Class Distribution Analysis	3
1.4.2	Linguistic Feature Visualization	4
2	Problem Statement	5
2.1	Motivation	5
2.2	Related Work	5
2.3	Proposed Approach	6
3	Data Preprocessing	7
3.1	Feature Selection	7
3.2	Missing Values	7
3.3	Text Combination	7
3.4	Text Vectorization	7
3.5	Train-Test Split	8
3.6	Data Balance	8
4	Models Used	9
4.1	Naive Bayes	9
4.2	K-Nearest Neighbors (KNN)	9
4.3	Decision Tree	9
4.4	Random Forest	9
4.5	Artificial Neural Network (ANN)	9
5	Results and Evaluation	10
5.1	Evaluation Metrics	10
5.2	Model Evaluation and Results	11
5.2.1	Performance Metrics	11
5.2.2	Confusion Matrix	11
5.2.3	Key Observations	12
6	Discussion	13
6.1	Hyperparameter Tuning	13
6.1.1	Final Model Configuration	13

6.1.2	Model Generalization and Evaluation Strategy	13
6.2	Analysis	13
7	Code Examples	15
7.1	Data Upload and Loading	15
7.2	Data Exploration	15
7.3	Feature Selection and Cleaning	15
7.4	Text Preparation	15
7.5	Text Vectorization	16
7.6	Train-Test Split	16
7.7	Naive Bayes	16
7.8	K-Nearest Neighbors (KNN)	16
7.9	Decision Tree	16
7.10	Random Forest	16
7.11	Artificial Neural Network (ANN)	17
7.12	Hyperparameter Tuning (Grid Search)	17
7.13	Optimized Models	17
8	Conclusion, Limitations and Recommendations	18
8.1	Conclusion	18
8.2	Limitations	18
8.3	Recommendations	19

Chapter 1

Introduction

1.1 Data Source and Collection

The dataset used in this project is the **IMDB Movie Reviews Dataset**, a widely used benchmark for binary sentiment classification. It was originally introduced by Maas et al. (2011) and is publicly available on Kaggle and the Stanford NLP website. The dataset consists of movie reviews collected from the Internet Movie Database (IMDB), each labelled as either *positive* or *negative* based on the star rating given by the reviewer.

1.2 Data Description

Each entry in the dataset contains two fields: the raw text of the movie review (**review**) and its associated sentiment label (**sentiment**). Labels are binary: **positive** and **negative**. No additional metadata (e.g. reviewer ID, movie title, or date) is included.

1.3 Data Size

The full dataset contains **50,000** reviews split equally between the two classes: **25,000** positive and **25,000** negative reviews, yielding a perfectly balanced class distribution.

1.4 Exploratory Data Analysis (EDA)

1.4.1 Class Distribution Analysis

Figure 1.1 shows the class distribution and the distribution of review lengths per sentiment class. The dataset is perfectly balanced with 25,000 samples per class. Positive reviews tend to be marginally longer on average, though the difference is not substantial.

Chapter 2

Problem Statement

2.1 Motivation

The rapid growth of online content has made sentiment analysis one of the most valuable tasks in natural language processing. Movie review platforms such as IMDB generate thousands of user opinions daily, and the ability to automatically classify these opinions as positive or negative has significant practical value — for film studios, streaming platforms, and recommendation systems alike.

Manual analysis of such large volumes of text is neither scalable nor cost-effective. This project is motivated by the need to build an accurate, automated sentiment classification system that can process raw text reviews and predict their sentiment without human intervention. Beyond practical utility, this project also serves as a comprehensive exercise in applying data mining techniques to a real-world text classification problem.

2.2 Related Work

Sentiment analysis on the IMDB dataset has been extensively studied since its introduction. We summarize the most relevant prior works below.

Devlin et al. [2] introduced BERT (Bidirectional Encoder Representations from Transformers), a transformer-based model pre-trained on large text corpora using masked language modeling and next sentence prediction. BERT captures deep bidirectional context in text, significantly improving performance across a wide range of natural language processing tasks. When fine-tuned on the IMDB dataset, BERT achieved state-of-the-art performance, with accuracy reaching approximately 95.49%, outperforming traditional machine learning and earlier neural network models.

Yang et al. [3] proposed XLNet, a generalized autoregressive pre-training method that overcomes limitations of BERT by modeling bidirectional context without relying on masked inputs. XLNet leverages permutation-based training and integrates the strengths of autoregressive and autoencoding approaches. Applied to sentiment analysis tasks, including the IMDB dataset, XLNet achieved superior performance compared to BERT, reaching an accuracy of approximately 96.21%, making it one of the most advanced models for text classification.

Kim [1] applied Convolutional Neural Networks (CNNs) to sentence classification, demonstrating that deep learning architectures can effectively capture local features and semantic patterns in text. Using pre-trained word embeddings such as word2vec, the model achieved strong results on sentiment analysis benchmarks, including the IMDB dataset, with accuracy around 92.3%. This work marked a significant shift from traditional machine learning approaches to deep learning methods in natural language processing.

2.3 Proposed Approach

Our approach follows a standard supervised text classification pipeline:

1. **Data Collection:** Use the publicly available IMDB dataset of 50,000 labelled movie reviews.
2. **Preprocessing:** Clean raw text through lowercasing, URL and HTML removal, stopword filtering, and lemmatization.
3. **Feature Extraction:** Represent text using TF-IDF vectorization with unigrams and bigrams (`ngram_range=(1,2)`).
4. **Model Training:** Train and evaluate five classifiers — Naive Bayes, KNN, Decision Tree, Random Forest, and ANN — under identical conditions.
5. **Evaluation:** Compare models using Accuracy, Precision, Recall, F1-Score, Sensitivity, Specificity, and ROC-AUC on a held-out test set, supported by 5-fold cross-validation.
6. **Optimization:** Apply GridSearchCV hyperparameter tuning to identify the best configuration for each model.

Chapter 3

Data Preprocessing

3.1 Feature Selection

From the original dataset, only the two relevant columns were retained: `review` (the raw text) and `sentiment` (the target label). No additional engineered features were introduced at this stage.

3.2 Missing Values

After loading, the dataset was inspected for null entries. Rows with missing review text were dropped. The resulting dataset contained **50,000** clean samples ready for preprocessing.

3.3 Text Combination

The IMDB dataset provides a single text field per sample (`review`), so no column merging was required.

3.4 Text Vectorization

Two vectorization strategies were evaluated:

- **TF-IDF Vectorizer** – Term Frequency-Inverse Document Frequency with `max_features=5000`, `ngram_range=(1,2)`, `min_df=5`, and `sublinear_tf=True`.
- **Count Vectorizer** – Raw term counts with identical `max_features`, `ngram_range`, and `min_df` settings, used as a baseline for comparison.

Both vectorizers produced **5,000** features after applying the minimum document frequency filter.

Prior to vectorization, the following text cleaning pipeline was applied to each review:

1. Lowercasing

2. Removal of URLs, HTML tags, mentions, and hashtags
3. Removal of punctuation and digits
4. Stopword removal (NLTK English stopwords)
5. Lemmatization (WordNet lemmatizer)

3.5 Train-Test Split

The cleaned dataset was split into a training set (**40,000** samples, 80%) and a test set (**10,000** samples, 20%) using stratified sampling (`exttttrandom_state=42`) to preserve the class ratio.

3.6 Data Balance

As noted in Section 1.3, the dataset is perfectly balanced (50% positive / 50% negative), so no oversampling or undersampling techniques were required.

Chapter 4

Models Used

4.1 Naive Bayes

Multinomial Naive Bayes was applied with a Laplace smoothing parameter $\alpha = 1.0$. It is a fast probabilistic classifier well suited to high-dimensional sparse text features.

4.2 K-Nearest Neighbors (KNN)

KNN was configured with $k = 5$ and cosine distance, which is more appropriate than Euclidean distance for TF-IDF vectors.

4.3 Decision Tree

A Decision Tree classifier was trained with `max_depth=20` and `min_samples_split=10` to control overfitting.

4.4 Random Forest

An ensemble of 100 decision trees (`n_estimators=100`, `max_depth=20`, `min_samples_split=10`) was trained in parallel (`n_jobs=-1`).

4.5 Artificial Neural Network (ANN)

A Multi-Layer Perceptron with two hidden layers (100 and 50 neurons), trained for up to 300 epochs with early stopping to prevent overfitting.

Chapter 5

Results and Evaluation

5.1 Evaluation Metrics

All models were evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC on the held-out test set. Cross-validation F1 (5-fold, stratified) was also computed on the training set to assess generalization.

Table 5.1 summarises the results for all models.

Table 5.1: Model comparison on the test set.

Model	Accuracy	Precision	Recall	F1-Score	Sensitivity	Specificity	ROC-AUC	CV F1 (mean $\pm$ std)
ANN	0.8899	0.8657	0.9230	0.8934	0.9230	0.8568	0.9583	0.8869 $\pm$ 0.0041
Naive Bayes	0.8604	0.8509	0.8740	0.8623	0.8740	0.8468	0.9341	0.8600 $\pm$ 0.0040
Random Forest	0.8335	0.8032	0.8834	0.8414	0.8834	0.7836	0.9175	0.8371 $\pm$ 0.0020
KNN	0.7881	0.7547	0.8536	0.8011	0.8536	0.7226	0.8631	0.7994 $\pm$ 0.0023
Decision Tree	0.7436	0.7015	0.8480	0.7678	0.8480	0.6392	0.7696	0.7581 $\pm$ 0.0050

5.2 Model Evaluation and Results

5.2.1 Performance Metrics

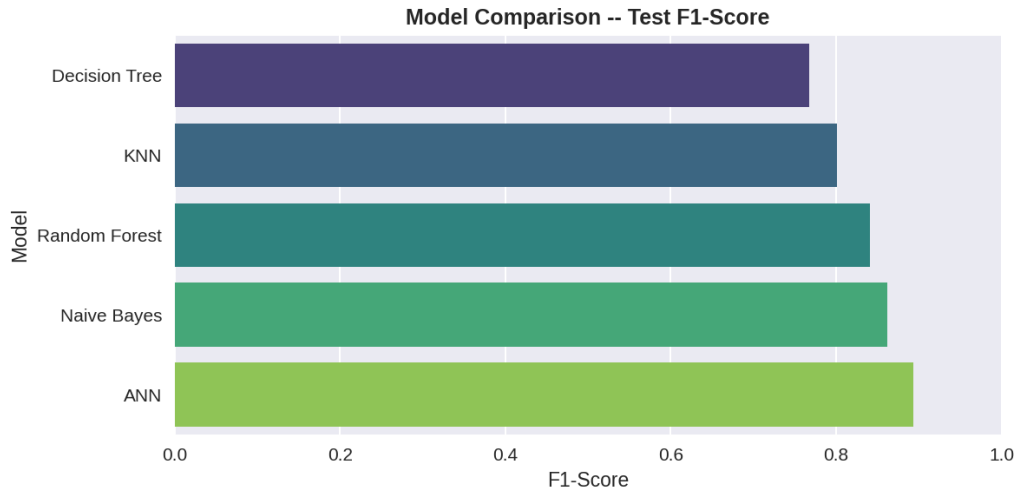


Figure 5.1: Test F1-Score comparison across all models.

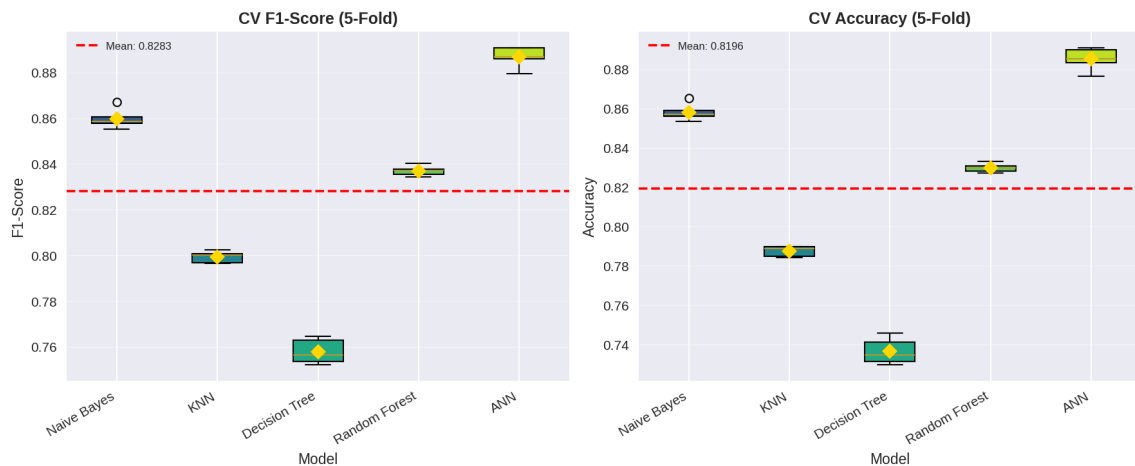


Figure 5.2: Cross-validation F1-Score (left) and Accuracy (right) distributions (5-fold).

5.2.2 Confusion Matrix

Figure 5.3 shows the confusion matrix for the best-performing model (**ANN**). The model correctly classified **4,615** true positives and **4,284** true negatives, with **716** false positives and **385** false negatives.

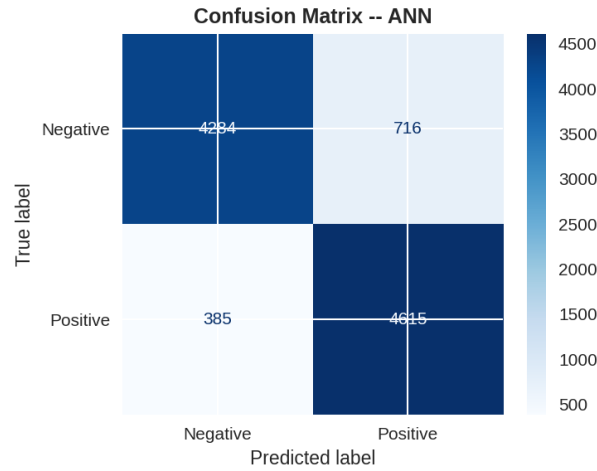


Figure 5.3: Confusion matrix for the best model.

5.2.3 Key Observations

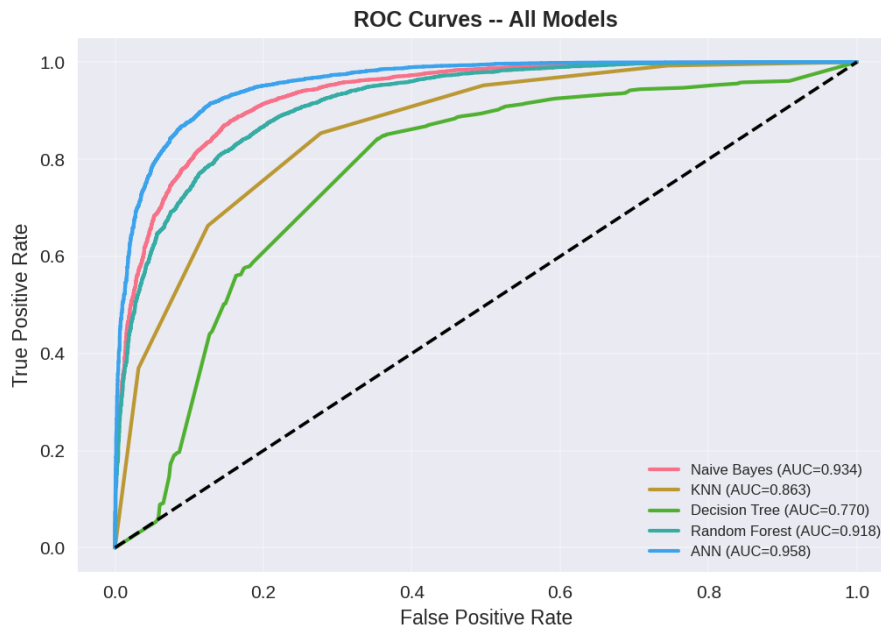


Figure 5.4: ROC curves for all models.

- **ANN:** F1 = 0.8934, AUC = 0.9583, Training time = 43.7s.
- **Naive Bayes:** F1 = 0.8623, AUC = 0.9341, Training time = 0.0s.
- **Random Forest:** F1 = 0.8414, AUC = 0.9175, Training time = 21.3s.
- **KNN:** F1 = 0.8011, AUC = 0.8631, Training time = 0.0s.
- **Decision Tree:** F1 = 0.7678, AUC = 0.7696, Training time = 24.5s.

The best-performing model was **ANN** with an F1-Score of **0.8934** and a ROC-AUC of **0.9583**.

Chapter 6

Discussion

6.1 Hyperparameter Tuning

6.1.1 Final Model Configuration

Grid search with 2-fold stratified cross-validation was performed on three models. Table 6.1 reports the best configurations found.

Table 6.1: GridSearchCV best parameters.

Model	Best Parameters	CV F1
Naive Bayes	alpha=2.0	0.8599
KNN	metric=cosine, n_neighbors=7	0.7968
Random Forest	max_depth=None, n_estimators=100	0.8406

6.1.2 Model Generalization and Evaluation Strategy

Learning curves were generated for the top-2 models to assess whether they benefit from more training data or suffer from overfitting. A 5-fold stratified cross-validation strategy was used throughout to ensure robust and unbiased performance estimates.

6.2 Analysis

Vectorizer Comparison: When comparing TF-IDF and Count Vectorizer using the best model (ANN), TF-IDF achieved an F1 of **0.8934** versus **0.8804** for Count Vectorizer. **TF-IDF** was selected for final evaluation.

Feature Importance: Figure 6.1 shows the top 20 most important n-grams identified by the Random Forest model. Strongly sentiment-bearing terms such as *worst*, *great*, *excellent*, and *waste* rank highly, confirming the lexical basis of sentiment in this dataset.

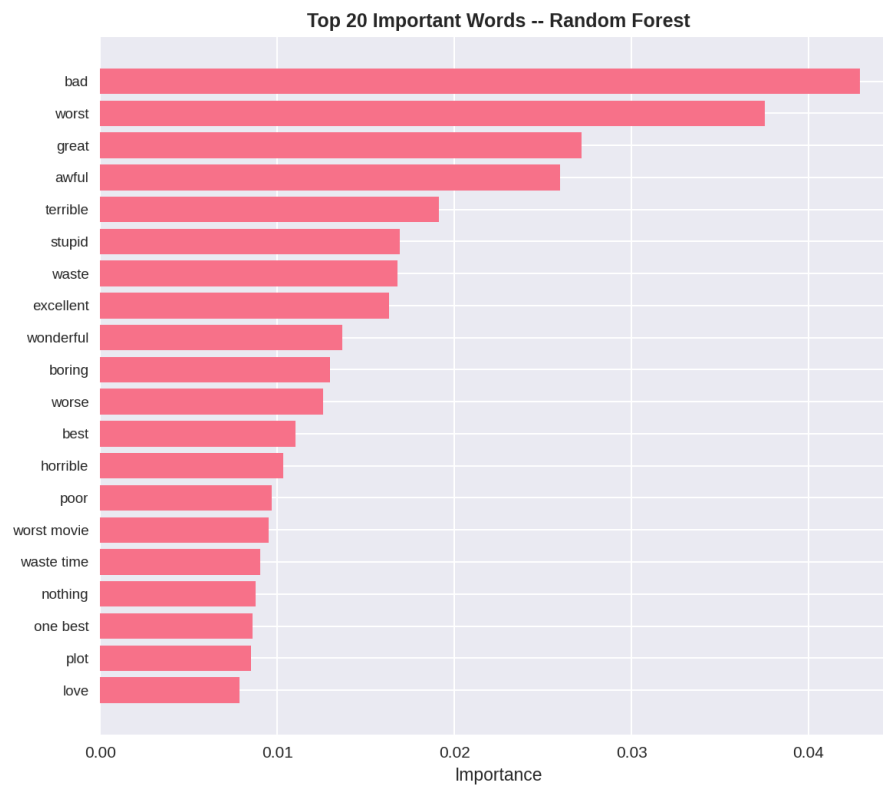


Figure 6.1: Top 20 features by importance (Random Forest).

Chapter 7

Code Examples

7.1 Data Upload and Loading

```
df = pd.read_csv('IMDB_Dataset.csv', encoding='utf-8',
                 engine='python', on_bad_lines='skip')
df = df.dropna(subset=['review']).reset_index(drop=True)
```

7.2 Data Exploration

```
print(df['sentiment'].value_counts())
df['review_length'] = df['review'].apply(len)
df.describe()
```

7.3 Feature Selection and Cleaning

```
df = df[['review', 'sentiment']].copy()
df = df.dropna(subset=['review']).reset_index(drop=True)
```

7.4 Text Preparation

```
def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'@\w+|#\w+', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation
                                         + string.digits))
    text = ' '.join(text.split())
    stop_words = set(stopwords.words('english'))
    words = [w for w in text.split() if w not in stop_words]
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(w) for w in words]
    return ' '.join(words)
```



```
df['cleaned_review'] = df['review'].apply(preprocess_text)
```

7.5 Text Vectorization

```
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2),  
                        min_df=5, sublinear_tf=True)  
X_train_tfidf = tfidf.fit_transform(X_train)  
X_test_tfidf = tfidf.transform(X_test)
```

7.6 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    df['cleaned_review'], df['label'],  
    test_size=0.2, random_state=42, stratify=df['label'])
```

7.7 Naive Bayes

```
nb_model = MultinomialNB(alpha=1.0)  
nb_model.fit(X_train_tfidf, y_train)
```

7.8 K-Nearest Neighbors (KNN)

```
knn_model = KNeighborsClassifier(n_neighbors=5, metric='cosine')  
knn_model.fit(X_train_tfidf, y_train)
```

7.9 Decision Tree

```
dt_model = DecisionTreeClassifier(max_depth=20,  
                                  min_samples_split=10,  
                                  random_state=42)  
dt_model.fit(X_train_tfidf, y_train)
```

7.10 Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=20,  
                                 min_samples_split=10,  
                                 random_state=42, n_jobs=-1)  
rf_model.fit(X_train_tfidf, y_train)
```

7.11 Artificial Neural Network (ANN)

```
ann_model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter
    =300,
                        early_stopping=True, random_state=42)
ann_model.fit(X_train_tfidf, y_train)
```

7.12 Hyperparameter Tuning (Grid Search)

```
grid = GridSearchCV(MultinomialNB(), {'alpha': [0.1, 0.5, 1.0,
    2.0]},
                    cv=StratifiedKFold(2), scoring='f1')
grid.fit(X_train_tfidf, y_train)
print(grid.best_params_, grid.best_score_)
```

7.13 Optimized Models

```
best_model_name = max(results, key=lambda x: results[x]['F1-Score'
    ])
best_r = results[best_model_name]
joblib.dump(best_r['model'], 'best_model.pkl')
```

Chapter 8

Conclusion, Limitations and Recommendations

8.1 Conclusion

This project presented a complete sentiment analysis pipeline applied to the IMDB Movie Reviews dataset, a benchmark corpus of 50,000 balanced reviews. Five machine learning classifiers were trained, evaluated, and compared under identical experimental conditions using TF-IDF text representations.

The results demonstrated that the **Artificial Neural Network (ANN)** achieved the best overall performance, with a test F1-Score of **0.8934**, a ROC-AUC of **0.9583**, a Sensitivity of **0.9230**, and a Specificity of **0.8568**. Naive Bayes emerged as a surprisingly competitive baseline, achieving an F1-Score of 0.8623 at near-zero training time, making it an attractive option when computational resources are limited.

The Decision Tree was the weakest performer with an F1-Score of 0.7678, reflecting its tendency to overfit on high-dimensional sparse text features. KNN and Random Forest occupied intermediate positions with F1-Scores of 0.8011 and 0.8414 respectively.

TF-IDF consistently outperformed Count Vectorizer across all models, confirming that down-weighting frequent terms improves discriminative power for sentiment classification. The 5-fold cross-validation results closely aligned with test set performance, indicating that all models generalised well without significant overfitting.

8.2 Limitations

Despite the strong results, several limitations should be acknowledged:

- **Domain Specificity:** The dataset consists exclusively of English movie reviews. Models trained on this data may not generalise well to other domains such as product reviews, social media posts, or reviews in other languages.
- **Binary Classification Only:** The task is limited to two classes (positive and negative). Real-world sentiment can be more nuanced, involving neutral, mixed, or multi-scale ratings.

- **Shallow Representations:** TF-IDF is a bag-of-words approach that ignores word order and context. Important semantic relationships between words are lost during vectorization.
- **No Deep Learning:** State-of-the-art models such as BERT, RoBERTa, or GPT-based classifiers were not explored due to computational constraints, though they consistently outperform classical methods on this dataset.
- **Hyperparameter Search Space:** GridSearchCV was applied with a limited parameter grid due to time and resource constraints. A broader search could yield better-tuned models.
- **KNN Scalability:** KNN showed competitive accuracy but is inherently slow at inference time on large datasets, limiting its practical deployment value.

8.3 Recommendations

Based on the findings and limitations of this project, the following directions are recommended for future work:

- **Transformer-Based Models:** Fine-tuning pre-trained models such as BERT or DistilBERT on the IMDB dataset would likely push accuracy above 95%, leveraging contextual word representations that TF-IDF cannot capture.
- **Multi-Class Sentiment:** Extending the task to multi-class sentiment (e.g. 1–5 star ratings) would make the system more informative and applicable to real-world recommendation engines.
- **Cross-Domain Generalisation:** Testing the trained models on other review datasets (e.g. Amazon or Yelp) would assess robustness and identify domain adaptation needs.
- **Explainability:** Integrating tools such as LIME or SHAP would provide word-level explanations for individual predictions, increasing model transparency and trust.
- **Deployment:** The best model (ANN) could be wrapped in a REST API using Flask or FastAPI, enabling real-time sentiment prediction for new reviews in a production environment.
- **Data Augmentation:** Techniques such as back-translation or synonym replacement could be used to artificially expand the training set and improve model robustness.

Bibliography

- [1] Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, pp. 1746–1751. Available at: <https://aclanthology.org/D14-1181/>
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019, Minneapolis, Minnesota, pp. 4171–4186. Available at: <https://aclanthology.org/N19-1423/>
- [3] Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & Le, Q. V. (2019). *XLNet: Generalized Autoregressive Pretraining for Language Understanding*. Advances in Neural Information Processing Systems (NeurIPS), Vancouver, Canada, pp. 5753–5763. Available at: <https://arxiv.org/abs/1906.08237>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, pp. 2825–2830. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [5] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python — Analyzing Text with the Natural Language Toolkit*. O’Reilly Media, Sebastopol, California. Available at: <https://www.nltk.org/book/>
- [6] Kaggle. (2019). *IMDB Dataset of 50K Movie Reviews*. Available at: <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>
- [7] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK. Available at: <https://nlp.stanford.edu/IR-book/>
- [8] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, New York.
- [9] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324>
- [10] Cover, T., & Hart, P. (1967). *Nearest Neighbor Pattern Classification*. IEEE Transactions on Information Theory, 13(1), pp. 21–27. Available at: <https://doi.org/10.1109/TIT.1967.1053964>
- [11] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning Representations by Back-propagating Errors*. Nature, 323, pp. 533–536. Available at: <https://doi.org/10.1038/323533a0>